

Problem Statement: In the secure view, Postgres decorrelates correlated queries, whereas in the view, it does not decorrelate them because that view can reveal hidden data points over time.

Goal: The first goal is to run queries and, by looking at the transformation, see how the secure view is decorrelating queries and how the view is still correlating them and may reveal information.

Experiment 1: Decorrelation Behaviour — Secure View vs. Plain View (Leakproof Predicate)

Objective: To compare how PostgreSQL's query planner handles a correlated subquery when accessed through a plain view versus a security_barrier view, using a leakproof filter condition.

Setup: Using the TPC-H customer table (200MB scale), a correlated query was defined that selects customers whose account balance exceeds the average balance of their own nation:

```
SELECT c1.c_custkey, c1.c_name, c1.c_nationkey, c1.c_acctbal
FROM customer c1
WHERE c1.c_acctbal > (
  SELECT AVG(c2.c_acctbal)
  FROM customer c2
  WHERE c2.c_nationkey = c1.c_nationkey
)
```

This query was wrapped in two views:

- plain_v — an ordinary view
- secure_v — created WITH (security_barrier)

Both views were queried with the same external filter:

```
SELECT * FROM plain_v WHERE c_nationkey = 5;
SELECT * FROM secure_v WHERE c_nationkey = 5;
```

Results: EXPLAIN (ANALYZE, VERBOSE) produced **identical query plans** for both views:

- Both used a Seq Scan on customer c1 with the filter c_nationkey = 5 AND c_acctbal > SubPlan 1.
- The correlated subquery ran as SubPlan 1, re-executed once per outer row (loops = 1195), i.e. **not decorrelated into a join** in either case.
- Execution time was nearly identical (~3.11s vs ~3.17s).

Observation:

1. The equality condition c_nationkey = 5 is a **leakproof** operator, so it is permitted to push down through the security_barrier fence just as freely as in the plain view — explaining why no difference was observed here.
2. More significantly, PostgreSQL did **not decorrelate the scalar correlated subquery (with aggregate) into a join in either view** — this is a limitation of the planner's decorrelation support in general, independent of security_barrier.

Conclusion (so far): This test establishes a baseline: for leakproof predicates, secure and plain views behave identically. The real divergence, if any, is expected to appear when the outer filter condition uses a **non-leakproof function** (e.g., LIKE, custom functions) — the next planned experiment — where security_barrier should block pushdown while the plain view does not.

Plan for normal view:

```
mpch-200mb=# EXPLAIN (ANALYZE, VERBOSE)
SELECT * FROM plain_v WHERE c_nationkey = 5;
               QUERY PLAN
-----
Seq Scan on public.customer c1  (cost=0.00..32941545.00 rows=398 width=33) (actual time=15.152..3111.983 rows=599.00
loops=1)
  Output: c1.c_custkey, c1.c_name, c1.c_nationkey, c1.c_acctbal
  Filter: ((c1.c_nationkey = 5) AND (c1.c_acctbal > (SubPlan 1)))
  Rows Removed by Filter: 29401
  Buffers: shared hit=861120
  SubPlan 1
    -> Aggregate  (cost=1098.00..1098.01 rows=1 width=32) (actual time=2.601..2.601 rows=1.00 loops=1195)
      Output: avg(c2.c_acctbal)
      Buffers: shared hit=860400
      -> Seq Scan on public.customer c2  (cost=0.00..1095.00 rows=1200 width=6) (actual time=0.001..2.472
rows=1195.00 loops=1195)
        Output: c2.c_custkey, c2.c_name, c2.c_address, c2.c_nationkey, c2.c_phone, c2.c_acctbal, c2.c_mktsegment,
c2.c_comment
        Filter: (c2.c_nationkey = c1.c_nationkey)
        Rows Removed by Filter: 28805
        Buffers: shared hit=860400
Planning:
  Buffers: shared hit=8
Planning Time: 0.275 ms
Execution Time: 3112.081 ms
(18 rows)
```

Secure View:

```
mpch-200mb=# EXPLAIN (ANALYZE, VERBOSE)
SELECT * FROM secure_v WHERE c_nationkey = 5;
               QUERY PLAN
-----
Seq Scan on public.customer c1  (cost=0.00..32941545.00 rows=398 width=33) (actual time=18.265..3172.085 rows=599.00
loops=1)
  Output: c1.c_custkey, c1.c_name, c1.c_nationkey, c1.c_acctbal
  Filter: ((c1.c_nationkey = 5) AND (c1.c_acctbal > (SubPlan 1)))
  Rows Removed by Filter: 29401
  Buffers: shared hit=861120
  SubPlan 1
    -> Aggregate  (cost=1098.00..1098.01 rows=1 width=32) (actual time=2.651..2.651 rows=1.00 loops=1195)
      Output: avg(c2.c_acctbal)
      Buffers: shared hit=860400
      -> Seq Scan on public.customer c2  (cost=0.00..1095.00 rows=1200 width=6) (actual time=0.001..2.523
rows=1195.00 loops=1195)
        Output: c2.c_custkey, c2.c_name, c2.c_address, c2.c_nationkey, c2.c_phone, c2.c_acctbal, c2.c_mktsegment,
c2.c_comment
        Filter: (c2.c_nationkey = c1.c_nationkey)
        Rows Removed by Filter: 28805
        Buffers: shared hit=860400
Planning:
  Buffers: shared hit=3
Planning Time: 0.275 ms
Execution Time: 3172.177 ms
(18 rows)
```

Experiment 2: Decorrelation Behavior — Secure View vs. Plain View (Non-Leakproof Predicate)

Objective: To test whether PostgreSQL's security_barrier restriction changes plan structure and performance when the outer filter uses a **non-leakproof** operator (LIKE), compared to a plain view.

Setup: Same views as Experiment 1 (plain_v, secure_v), now queried with:

```
SELECT * FROM plain_v WHERE c_name LIKE 'A%';  
  
SELECT * FROM secure_v WHERE c_name LIKE 'A%';
```

Results:

Metric	plain_v	secure_v
Plan shape	Single Seq Scan, both filters merged	Subquery Scan wrapping a separate inner Seq Scan
SubPlan execution	Never executed (0 loops)	Executed 30,000 times (once per row)
Buffers	720	21,600,720
Execution Time	14 ms	77,232 ms (~77 sec)

Observation:

1. In `plain_v`, the planner merged the LIKE filter and the correlated SubPlan into a single scan and evaluated them with short-circuit AND logic. Since `c_name LIKE 'A%'` matched zero rows, the planner evaluated it first and the correlated subquery was **never executed** — enormous savings.
2. In `secure_v`, the `security_barrier` fence forced the view's internal query (with the correlated subquery) to execute and materialize **completely first**, as an isolated Subquery Scan, before the outer LIKE filter could be applied. This is because LIKE is not leakproof and cannot be pushed inside the barrier. As a result, the correlated subquery ran once for **every** row (30,000 loops) regardless of whether that row would later be filtered out — a $\sim 5,500\times$ **slowdown** (14ms $\rightarrow$ 77s).

Conclusion: This experiment demonstrates the concrete cost of the `security_barrier` protection: it is not that the secure view fails to decorrelate differently than the plain view in isolation — neither view decorrelates the aggregate subquery into a join. Rather, the barrier prevents the planner from **reordering/merging filters** across the boundary, blocking optimizations like short-circuit evaluation and early filtering. The performance penalty comes from **forced early materialization of the entire correlated result set** before any non-leakproof condition can be applied, directly illustrating the security-vs-performance trade-off documented for `security_barrier` views.

```
mpch-200mb=# EXPLAIN (ANALYZE, VERBOSE)
```

```
SELECT * FROM plain_v WHERE c_name LIKE 'A%';
```

QUERY PLAN

```
Seq Scan on public.customer c1 (cost=0.00..32941545.00 rows=1 width=33) (actual time=13.622..13.624 rows=0.00 loops=1)
```

```
Output: c1.c_custkey, c1.c_name, c1.c_nationkey, c1.c_acctbal
```

```
Filter: (((c1.c_name)::text ~~ 'A% '::text) AND (c1.c_acctbal > (SubPlan 1)))
```

```
Rows Removed by Filter: 30000
```

```
Buffers: shared hit=720
```

```
SubPlan 1
```

```
-> Aggregate (cost=1098.00..1098.01 rows=1 width=32) (never executed)
```

```
Output: avg(c2.c_acctbal)
```

```
-> Seq Scan on public.customer c2 (cost=0.00..1095.00 rows=1200 width=6) (never executed)
```

```
Output: c2.c_custkey, c2.c_name, c2.c_address, c2.c_nationkey, c2.c_phone, c2.c_acctbal, c2.c_mktsegment, c2.c_comment
```

```
Filter: (c2.c_nationkey = c1.c_nationkey)
```

```
Planning Time: 0.627 ms
```

```
Execution Time: 14.001 ms
```

```
(13 rows)
```

mpch-200mb=# EXPLAIN (ANALYZE, VERBOSE)

SELECT * FROM secure_v WHERE c_name LIKE 'A%';

QUERY PLAN

Subquery Scan on secure_v (cost=0.00..32941595.00 rows=50 width=33) (actual time=77232.541..77232.542 rows=0.00 loops=1)

Output: secure_v.c_custkey, secure_v.c_name, secure_v.c_nationkey, secure_v.c_acctbal

Filter: ((secure_v.c_name)::text ~ 'A%')::text

Rows Removed by Filter: 14873

Buffers: shared hit=21600720

-> Seq Scan on public.customer c1 (cost=0.00..32941470.00 rows=10000 width=33) (actual time=23.280..77230.412 rows=14873.00 loops=1)

Output: c1.c_custkey, c1.c_name, c1.c_nationkey, c1.c_acctbal

Filter: (c1.c_acctbal > (SubPlan 1))

Rows Removed by Filter: 15127

Buffers: shared hit=21600720

SubPlan 1

-> Aggregate (cost=1098.00..1098.01 rows=1 width=32) (actual time=2.574..2.574 rows=1.00 loops=30000)

Output: avg(c2.c_acctbal)

Buffers: shared hit=21600000

-> Seq Scan on public.customer c2 (cost=0.00..1095.00 rows=1200 width=6) (actual time=0.003..2.444 rows=1200.84 loops=30000)

Output: c2.c_custkey, c2.c_name, c2.c_address, c2.c_nationkey, c2.c_phone, c2.c_acctbal, c2.c_mktsegment, c2.c_comment

Filter: (c2.c_nationkey = c1.c_nationkey)

Rows Removed by Filter: 28799

Buffers: shared hit=21600000

Planning:

Buffers: shared hit=3

Planning Time: 2.013 ms

Execution Time: 77232.901 ms

(23 rows)

Experiment 3: Non-Leakproof Predicate with Non-Zero Selectivity

Objective:

To isolate whether the massive slowdown seen in Experiment 2 was due to a genuine planner restriction, or just an artifact of the LIKE pattern matching zero rows. This experiment uses a pattern ('Customer#000012%') that matches a substantial, non-trivial set of rows (~1000 on the raw table).

Setup:

Same query as Experiment 2, with a higher-selectivity LIKE pattern:

```
SELECT * FROM plain_v WHERE c_name LIKE 'Customer#000012%';  
SELECT * FROM secure_v WHERE c_name LIKE 'Customer#000012%';
```

Results:

Metric	Raw table	plain_v	secure_v
Rows matched	1000	486	486
SubPlan loops	—	1000	30,000
Buffers	720	720,720	21,600,720
Execution Time	12 ms	2,578 ms (~2.6s)	87,447 ms (~87s)

Observation:

1. In plain_v, the planner merged the LIKE filter with the correlated SubPlan in a single Seq Scan with a combined AND filter. Postgres's short-circuit evaluation meant the correlated subquery only ran for rows that passed (or were being evaluated alongside) the LIKE filter — **1000 loops**, matching the row count of the LIKE predicate alone.
2. In secure_v, the Subquery Scan fence again forced the inner query (with the correlated SubPlan) to fully execute for **all 30,000 rows** *before* the

outer LIKE filter was applied — regardless of the LIKE predicate's actual selectivity.

3. This confirms the effect in Experiment 2 was **not a fluke of zero matches** — the same $\sim 30\times$ gap in SubPlan executions (1000 vs. 30,000) and resulting $\sim 34\times$ **slowdown** (2.6s $\rightarrow$ 87s) appears here with a realistic, non-trivial selectivity.

Conclusion:

Across both zero-match and non-zero-match cases, the pattern is consistent: `security_barrier` unconditionally forces full materialization of the correlated subquery for every row in the base relation, before any non-leakproof filter can be applied — independent of how selective that filter actually is. This confirms the fence is a **structural planner restriction**, not something whose cost varies with query selectivity; the penalty is paid regardless of how much the outer filter would have reduced the work.

```
tpch-200mb=# EXPLAIN ANALYZE SELECT * FROM plain_v WHERE c_name LIKE 'Customer#000012%';  
QUERY PLAN
```

```
Seq Scan on customer c1 (cost=0.00..32941545.00 rows=404 width=33) (actual time=45.583..2613.314 rows=486.00  
loops=1)
```

```
Filter: (((c_name)::text ~ 'Customer#000012% '::text) AND (c_acctbal > (SubPlan 1)))
```

```
Rows Removed by Filter: 29514
```

```
Buffers: shared hit=720720
```

```
SubPlan 1
```

```
-> Aggregate (cost=1098.00..1098.01 rows=1 width=32) (actual time=2.592..2.592 rows=1.00 loops=1000)
```

```
Buffers: shared hit=720000
```

```
-> Seq Scan on customer c2 (cost=0.00..1095.00 rows=1200 width=6) (actual time=0.003..2.462 rows=1200.83  
loops=1000)
```

```
Filter: (c_nationkey = c1.c_nationkey)
```

```
Rows Removed by Filter: 28799
```

```
Buffers: shared hit=720000
```

```
Planning:
```

```
Buffers: shared hit=73
```

```
Planning Time: 8.356 ms
```

```
Execution Time: 2614.394 ms
```

```
(15 rows)
```

mpch-200mb=# EXPLAIN ANALYZE SELECT * FROM secure_v WHERE c_name LIKE 'Customer#000012%';

QUERY PLAN

Subquery Scan on secure_v (cost=0.00..32941595.00 rows=50 width=33) (actual time=31478.944..77336.252 rows=486.00 loops=1)

Filter: ((secure_v.c_name)::text ~ 'Customer#000012%':::text)

Rows Removed by Filter: 14387

Buffers: shared hit=21600720

-> Seq Scan on customer c1 (cost=0.00..32941470.00 rows=10000 width=33) (actual time=25.254..77333.188 rows=14873.00 loops=1)

Filter: (c_acctbal > (SubPlan 1))

Rows Removed by Filter: 15127

Buffers: shared hit=21600720

SubPlan 1

-> Aggregate (cost=1098.00..1098.01 rows=1 width=32) (actual time=2.577..2.577 rows=1.00 loops=30000)

Buffers: shared hit=21600000

-> Seq Scan on customer c2 (cost=0.00..1095.00 rows=1200 width=6) (actual time=0.003..2.447 rows=1200.84 loops=30000)

Filter: (c_nationkey = c1.c_nationkey)

Rows Removed by Filter: 28799

Buffers: shared hit=21600000

Planning:

Buffers: shared hit=6

Planning Time: 0.581 ms

Execution Time: 77337.536 ms

(19 rows)

Now the next step is to find the reasons in the code:

1: Before predicate pushdown is even considered, the planner attempts to pull up subqueries to flatten the join tree.

Where: optimizer/prep/prepjointree.c: L1873 in `is_simple_subquery()`.

Logic: If `rte->security_barrier` is true, `is_simple_subquery()` immediately returns false.

Effect: This acts as a planner fence. It prevents the subquery/view from being flattened/pulled up into the parent query, forcing Postgres to plan the subquery separately and evaluate it using a Subquery Scan plan node. This keeps the view's boundaries intact.

```
src > backend > optimizer > prep > C prepjointree.c > is_simple_subquery(PlannerInfo *, Query *, RangeTblEntry *, JoinExpr *)
1827 JoinExpr *lowest_outer_join)
1868 /*
1869  * Don't pull up if the RTE represents a security-barrier view; we
1870  * couldn't prevent information leakage once the RTE's Vars are scattered
1871  * about in the upper query.
1872  */
1873 if (rte->security_barrier)
1874     return false;
1875
```

Reason 2: Deciding Whether to Push Down Predicates

For the quals (WHERE clauses) defined in the outer query, the planner checks if they can be safely pushed down inside the subquery scan.

Where: optimizer/path/allpaths.c: L2575 and L3943 in `set_subquery_pathlist()` and `qual_is_pushdown_safe()`.

Logic:

In `set_subquery_pathlist()`, the optimizer populates a `pushdown_safety_info` struct:

```
safetyInfo.unsafeLeaky = rte->security_barrier;
```

For each restriction clause of the relation, it calls `qual_is_pushdown_safe()`.

If `safetyInfo->unsafeLeaky` is true (due to the security barrier view), it checks if the qual contains any leaky functions/operators:

```
if (safetyInfo->unsafeLeaky && contain_leaked_vars(qual))  
    return PUSHDOWN_UNSAFE;
```

Effect: If the qual is determined to contain any leaky references/functions (PUSHDOWN_UNSAFE), it is not pushed down for evaluation during the subquery scan. Instead, it gets evaluated as a filter (qual) above the Subquery Scan node, ensuring the view's secure filters execute first.

```
src > backend > optimizer > path > allpaths.c > set_subquery_pathlist(PlannerInfo *, RelOptInfo *, Index, RangeTblEntry *)  
2530      Index rti, RangeTblEntry *rte)  
2567  
2568      /*  
2569       * If the subquery has the "security_barrier" flag, it means the subquery  
2570       * originated from a view that must enforce row-level security. Then we  
2571       * must not push down quals that contain leaky functions. (Ideally this  
2572       * would be checked inside subquery_is_pushdown_safe, but since we don't  
2573       * currently pass the RTE to that function, we must do it here.)  
2574       */  
2575      safetyInfo->unsafeLeaky = rte->security_barrier;
```

```
src > backend > optimizer > path > allpaths.c > qual_is_pushdown_safe(Query *, Index, RestrictInfo *, pushdown_safety_info *)  
3926      pushdown_safety_info *safetyInfo)  
3941  
3942      /* Refuse leaky quals if told to (point 3) */  
3943      if (safetyInfo->unsafeLeaky &&  
3944          contain_leaked_vars(qual))  
3945          return PUSHDOWN_UNSAFE;
```

Reason 3: Checking for Leaky Functions/Operators (Leakproofness)

To evaluate `contain_leaked_vars()`, Postgres walks the query tree looking for expressions where local variables are passed to non-leakproof functions.

Where: `optimizer/util/clauses.c:L1305-1465` in `contain_leaked_vars()` and `contain_leaked_vars_walker()`.

Logic:

It recursively traverses the AST representation of the qual.

When it encounters function calls or operators (e.g. `FuncExpr`, `OpExpr`, `ScalarArrayOpExpr`), it looks up the function OID.

It calls `get_func_leakproof(func_id)` which checks the `proleakproof` boolean column in the `pg_proc` system catalog.

If the function/operator is not leakproof (e.g. standard operators like LIKE / ~~ do not have proleakproof = true) and is passed a Var from the relation, the walker immediately returns true (meaning it is a leaky qualification).

If the comparison is leakproof (e.g., standard = integer comparison like c_nationkey = 5 is marked leakproof), it does not flag a leak, allowing the pushdown.

```
/*
 * get_func_leakproof
 *     Given procedure id, return the function's leakproof field.
 */
bool
get_func_leakproof(Oid funcid)
{
    HeapTuple    tp;
    bool         result;

    tp = SearchSysCache1(PROC0ID, ObjectIdGetDatum(funcid));
    if (!HeapTupleIsValid(tp))
        elog(ERROR, "cache lookup failed for function %u", funcid);

    result = ((Form_pg_proc) GETSTRUCT(tp))->proleakproof;
    ReleaseSysCache(tp);
    return result;
}
```

4. Runtime Evaluation Order and Security Levels

If quals cannot be pushed down, or if multiple quals are evaluated at the same plan node, they must be ordered correctly to prevent information leakage.

Where: optimizer/plan/createplan.c:L5390 in order_qual_clauses().

Logic:

Each qual clause is assigned a security_level (assigned during preprocessing and distribution in process_security_barrier_quals).

Quals are sorted by their security_level first, and then by their estimated execution cost.

A crucial exception is made for leakproof quals: if a clause is marked leakproof and is cheap (estimated cost is less than $10 * \text{cpu_operator_cost}$), it is temporarily treated as having `security_level = 0`. This allows Postgres to safely sort it in front of security barrier quals to optimize performance without risking data leaks.

Findings:

Commit for security_barrier:

<https://github.com/postgres/postgres/commit/0e4611c0234d89e288a53351f775c59522baed7c>

Commit for securityQuals:

<https://github.com/postgres/postgres/commit/842faa714c0454d67e523f5a0b6df6500e9bc1a5>

This one is for RLS but mentions security_levels:

<https://github.com/postgres/postgres/commit/215b43cdc8d6b4a1700886a39df1ee735cb0274d>

Parallel_safe:

Thread safety:

Why Process and not thread:

<https://postgrespro.com/list/id/3A2B3CF2.88C3E55D@alumni.caltech.edu>

<https://www.postgresql.org/message-id/4180C6C7.1000708@mailblocks.com>

1. Decorrelation and Security

1. Understand how **query decorrelation/flattening can create a security hole**. (Incomplete)
2. Reproduce the **error-based value leakage** example suggested by GPT. (Incomplete)
3. Create concrete examples and test them on **PostgreSQL and MS SQL Server**. (Incomplete)
4. Check whether the **same security issue exists in both databases**. (Incomplete)

1. The first question is: can a secure view reveal data directly to a user?

Answer: No. This is a commit from a developer in 2011 suggesting secure views will not leak data points to users.

Commit :

<https://github.com/postgres/postgres/commit/0e4611c0234d89e288a53351f775c59522baed7c>

What the commit restricted: Prevents user-supplied functions/operators from being pushed down and executed against hidden rows during query execution/plan pushdown.

What still leaks (not covered by this commit):

1. **Optimizer statistics exposure** — the planner still consults `pg_statistic` (histograms, most-common-values lists) sampled from the *full* underlying table, even when the user only has access to a restricted view or row-security-filtered subset

Problem 1: Optimizer statistics exposure

Problem (first raised): Selectivity-estimation functions ran user-supplied operators directly over `pg_statistic` data with no security check at all — any operator, leaky or not, could be applied to statistics of columns the user had no privileges on.

Solution: Add a check before letting the estimator touch statistics: allow it only if (1) the user has table/column privilege on the underlying table, or (2) the operator is LEAKPROOF. Otherwise, planning proceeds as if no statistics exist.

Commit: `e2d4ef8de869c57e3bf270a30c12d48c2ce4e00c` — "Add security checks to selectivity estimation functions" (Peter Eisentraut / Tom Lane, 2017)

<https://git.postgresql.org/pg/commitdiff/e2d4ef8de869c57e3bf270a30c12d48c2ce4e00c>
(CVE-2017-7484)

2. **Leaky operator via cost estimation** — a crafted operator can be evaluated during *plan estimation* (not execution), a code path the `security_barrier` restriction doesn't cover, letting fragments of hidden statistics surface

Problem 2: Leaky operator via cost estimation

Problem: Same root cause as #1 — the leaky-operator path *is* the cost-estimation path (`selfuncs.c`). No separate first report; it's the same bug described above.

Solution: Same fix as #1 (the leakproof/privilege gate in `selfuncs.c`).

Commit: Same as above —
`e2d4ef8de869c57e3bf270a30c12d48c2ce4e00c`

3. **Bypasses view ACLs** — works even though the user has no grant on the underlying table (Invoker and definer problem).

CVE documenting this: <https://nvd.nist.gov/vuln/detail/CVE-2025-8713>

Already fixed: <https://www.postgresql.org/docs/17/release-17-6.html>

4. **Timing/plan-choice side channels** (acknowledged by the same commit's own docs, separate from the CVE) — EXPLAIN output and query runtime can reveal data volume/distribution even without any leaky operator

The problem, as raised

Laurenz Albe (Cybertec) noticed that with EXPLAIN (ANALYZE), a user can easily learn **how many rows** a row-level security (RLS) policy or a `security_barrier` view's filter condition removed — even though they never see the actual row contents. For example:

```
EXPLAIN (ANALYZE) SELECT * FROM sensitive_view WHERE
some_condition;
```

The plan output shows "rows removed by filter"-style numbers at each node. If a hidden RLS/view predicate filtered out, say, 4 of 10 rows, the user now knows something about the protected data's shape — even though `security_barrier`/RLS is supposed to prevent inference about hidden rows.

He proposed a patch to fix it: restrict EXPLAIN (ANALYZE) on such statements to members of the `pg_read_all_stats` role (the same role already trusted to see sensitive `pg_statistic` data), leaving ordinary users unable to run it against RLS/`security_barrier` queries.

The discussion — why it was ultimately rejected

Round 1 — Dean Rasheed's pushback: He agreed the row-count leak is real, but pointed out EXPLAIN (ANALYZE) isn't the only way to learn it — you can get similar information from `pg_stat_get_tuples_returned()`, `pg_class.rel tuples`, or even the row estimates in a plain EXPLAIN (no ANALYZE needed, no execution required). His conclusion: since you'd have to close all of those other paths too to actually fix the underlying issue, patching just EXPLAIN (ANALYZE) doesn't solve the real problem — it only closes one door among several.

Round 2 — Albe partially conceded, partially pushed back: He agreed `pg_stat_get_tuples_returned()` is a real alternate leak worth fixing too, but disputed the other two: `pg_class.rel tuples` is only a rough estimate that says nothing about actual contents, and a plain `EXPLAIN` gives you the planner's *estimate* of rows, not the actual count filtered by the RLS policy at runtime — so those two aren't equivalent leaks. Despite disagreeing on some of the specifics, he still maintained his patch was worth having.

Final resolution — the real reason it died: Albe took the question to the private `pgsql-security` list to get a more authoritative answer, and reported back that the consensus there was broader than "there are other side channels." The real argument was: **any user who can run arbitrary SQL against the database can already work around row-level security through other means, given enough determination** — so the marginal security value of blocking one specific inference channel (`EXPLAIN ANALYZE` row counts) is low, while the cost is real: it would make it harder for legitimate users to debug and tune query performance on `RLS/security_barrier` tables, since `EXPLAIN (ANALYZE)` is the standard tool for that. On that cost/benefit basis, the patch was marked **Rejected** in September 2024.

The core reasoning distilled

It's not that the maintainers deny the leak exists — Rasheed and Albe both agree it's real. The reasoning for *not* fixing it is:

1. It's one of several overlapping ways to infer filtered-row counts, not a unique hole — closing it alone gives a false sense of security.
2. `RLS/security_barrier` were never designed to resist a user who can execute arbitrary SQL and is specifically trying to extract information via inference — they protect against accidental exposure and straightforward queries, not a determined adversary probing timing/row-count side channels.
3. The fix's cost (breaking normal query-performance debugging) outweighs closing one narrow, already-non-exclusive channel.

Conclusion:

As of PostgreSQL's current patched versions ($\geq 17.6/16.10/15.14/14.19/13.22$), all publicly disclosed data-content leaks via security_barrier views/RLS have been fixed. The one channel PostgreSQL has explicitly declined to fix is row-count/cardinality inference via EXPLAIN (ANALYZE) and related statistics functions — a design trade-off, not an oversight. However, the 2017→2019→2025 pattern of recurring, related vulnerabilities in the same code path shows this is an actively contested boundary, not a closed problem — so "no other leak exists" should be read as "no other leak is currently known," not as a guarantee.

2. Threading and Correctness

1. Understand why **threading is related to correctness rather than security**. (Done)

<https://www.postgresql.org/docs/current/parallel-safety.html>

The PostgreSQL documentation does not flag any security concerns related to marking predicates as **parallel-safe** or **parallel-unsafe**.

2. Check when PostgreSQL allows or disallows threading for **flattened vs. correlated queries**. (Done)

Query 1: Which will get flattened

Query:: SELECT * FROM orders

WHERE o_orderkey IN (SELECT l_orderkey FROM lineitem WHERE
l_quantity > 45);

QUERY PLAN

```
-----  
-----  
Gather (cost=30402.03..49370.04 rows=115237 width=107) (actual  
time=138.427..190.890 rows=98918.00 loops=1)  
  Workers Planned: 2  
  Workers Launched: 2  
  Buffers: shared hit=662 read=27162  
    -> Parallel Hash Semi Join (cost=29402.03..36846.34 rows=48015  
width=107) (actual time=131.828..177.082 rows=32972.67 loops=3)  
      Hash Cond: (orders.o_orderkey = lineitem.l_orderkey)  
      Buffers: shared hit=662 read=27162  
        -> Parallel Seq Scan on orders (cost=0.00..6522.00 rows=125000  
width=107) (actual time=0.307..23.154 rows=100000.00 loops=3)  
          Buffers: shared hit=380 read=4892  
        -> Parallel Hash (cost=28801.84..28801.84 rows=48015 width=4) (actual  
time=131.333..131.333 rows=40033.67 loops=3)  
          Buckets: 131072 Batches: 1 Memory Usage: 5760kB  
          Buffers: shared hit=282 read=22270  
        -> Parallel Seq Scan on lineitem (cost=0.00..28801.84 rows=48015  
width=4) (actual time=0.341..121.804 rows=40033.67 loops=3)  
          Filter: (l_quantity > '45'::numeric)  
          Rows Removed by Filter: 359956  
          Buffers: shared hit=282 read=22270  
Planning Time: 3.701 ms  
Execution Time: 194.957 ms  
(18 rows)
```

Query 2:: Correlated:

```
SELECT c.c_custkey, c.c_name, (SELECT o.o_orderdate FROM orders o
WHERE o.o_custkey = c.c_custkey ORDER BY o.o_orderdate DESC LIMIT
1) AS last_order_date FROM customer c WHERE c.c_custkey <= 10;
```

QUERY PLAN

Seq Scan on customer c (cost=0.00..91315.83 rows=10 width=27) (actual
time=34.639..352.766 rows=10.00 loops=1)

Filter: (c_custkey <= 10)

Rows Removed by Filter: 29990

Buffers: shared hit=9223 read=44220

SubPlan 1

-> Limit (cost=9022.08..9022.08 rows=1 width=4) (actual
time=24.542..24.542 rows=0.70 loops=10)

Buffers: shared hit=8503 read=44220

-> Sort (cost=9022.08..9022.12 rows=16 width=4) (actual
time=24.538..24.538 rows=0.70 loops=10)

Sort Key: o.o_orderdate DESC

Sort Method: top-N heapsort Memory: 25kB

Buffers: shared hit=8503 read=44220

-> Seq Scan on orders o (cost=0.00..9022.00 rows=16 width=4)
(actual time=8.406..24.391 rows=9.70 loops=10)

Filter: (o_custkey = c.c_custkey)

Rows Removed by Filter: 299990

Buffers: shared hit=8500 read=44220

Planning:

Buffers: shared hit=11 read=2

Planning Time: 5.653 ms

Execution Time: 353.010 ms

3. Observe and document the **threading behavior in MS SQL Server**.
(Done)

Postgres is process-based, and each process (leader or worker) is single-threaded internally — with no threading inside a single process at all. Parallelism happens by running multiple such processes side-by-side, coordinated via inter-process communication. This works well for "split the data and do the same independent work on each slice" cases (scans, filters, hash joins), but becomes expensive when a plan needs to pass a per-row correlated value into a worker, so Postgres's planner just doesn't offer parallelism for that pattern.

SQL Server, by contrast, uses threads inside a single process, which share memory directly. This makes passing per-row correlated values across "workers" (threads) cheap, so SQL Server's optimizer can and does choose parallel plans even for correlated subqueries — something Postgres structurally avoids.